

Review

- Last Class:
 - Thread parallelism
- Today's class:
 - SIMD and vectorization

Chapter 4

Data-Level Parallelism in Vector, SIMD, and GPU Architectures.

Flynn's Taxonomy

<i>classic von Neumann</i> SISD Single instruction stream Single data stream	(SIMD) Single instruction stream Multiple data stream
MISD Multiple instruction stream Single data stream <i>Does it make sense? Yes, systolic array.</i>	(MIMD) Multiple instruction stream Multiple data stream

- Concurrency arises from performing the **same operations on different pieces of data**
 - Single instruction multiple data (SIMD)
 - E.g., dot product of two vectors
- Contrast with data flow
 - Concurrency arises from executing different operations in parallel (in a data driven manner)
- Contrast with thread (“control”) parallelism
 - Concurrency arises from executing different threads of control in parallel

- SIMD architectures can exploit significant data-level parallelism for:
 - matrix-oriented scientific computing
 - media-oriented image and sound processors
- SIMD is more energy efficient than MIMD
 - Only needs to fetch one instruction per data operation
 - Makes SIMD attractive for personal mobile devices
- SIMD allows programmer to continue to think sequentially

- Single instruction operates on multiple data elements
 - In time or in space
- Multiple processing elements
- Time-space duality
 - **Array processor**: Instruction operates on multiple data elements at the same time
 - **Vector processor**: Instruction operates on multiple data elements in consecutive time steps

Array vs. Vector Processors

Instruction Stream

```
LD   VR ← A[3:0]
ADD  VR ← VR, 1
MUL  VR ← VR, 2
ST   A[3:0] ← VR
```

Time

ARRAY PROCESSOR



Same op @ same time

LD0	LD1	LD2	LD3
AD0	AD1	AD2	AD3
MU0	MU1	MU2	MU3
ST0	ST1	ST2	ST3

Different ops @ same space

Space

VECTOR PROCESSOR



Different ops @ time

LD0			
LD1	AD0		
LD2	AD1	MU0	
LD3	AD2	MU1	ST0
	AD3	MU2	ST1
		MU3	ST2
			ST3

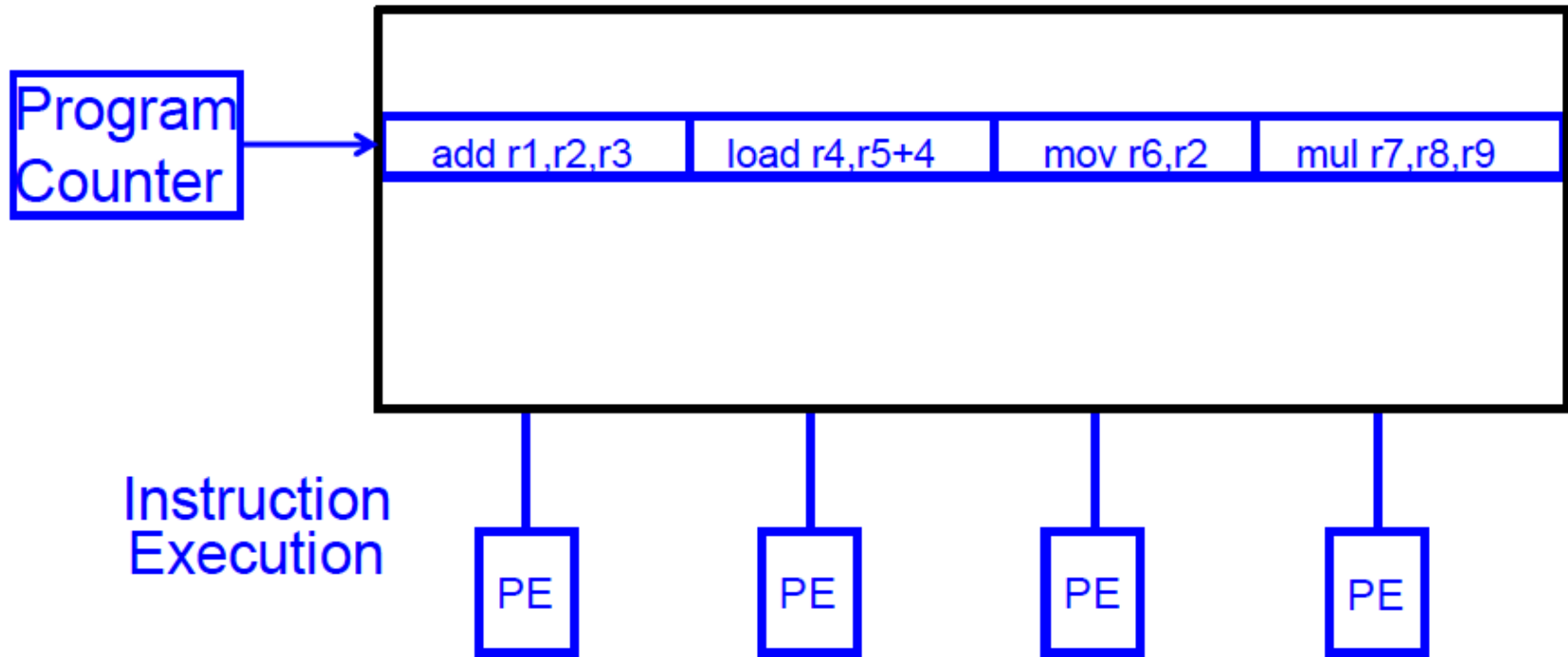
Same op @ space

Space

SIMD Array Processing vs. VLIW



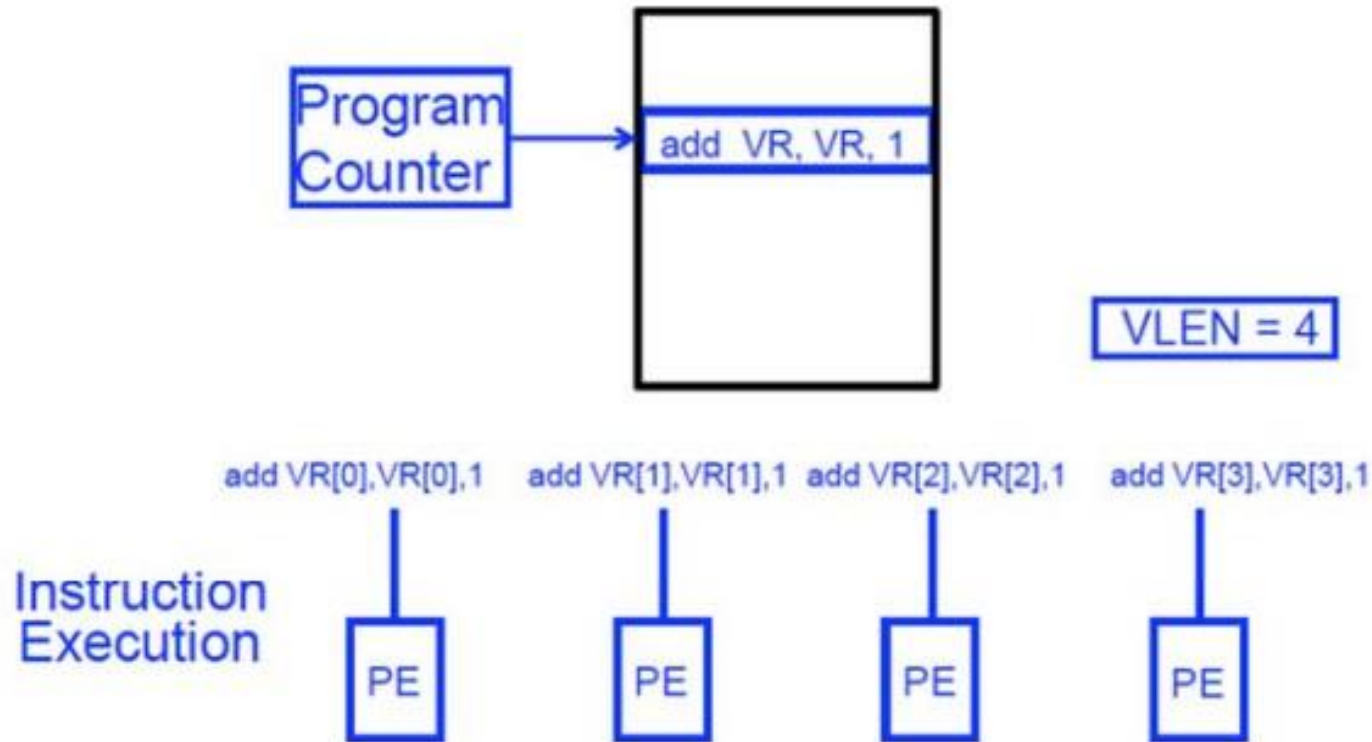
- VLIW (Very Large Instruction Word)



SIMD Array Processing vs. VLIW



- VLIW (Very Large Instruction Word)



Vector Processors

- A vector is a one-dimensional array of numbers
- Many scientific/commercial programs use vectors
 for (i = 0; i<=49; i++)
 C[i] = (A[i] + B[i]) / 2
- A vector processor is one whose instructions operate on vectors rather than scalar (single data) values
- Basic requirements
 - Need to load/store vectors → vector registers (contain vectors)
 - Need to operate on vectors of different lengths → vector length register (VLEN)
 - Elements of a vector might be stored apart from each other in memory → vector stride register (VSTR)
 - Stride: distance between two elements of a vector

- A vector instruction performs an operation on each element in consecutive cycles
 - Vector functional units are pipelined
 - Each pipeline stage operates on a different data element

- Vector instructions allow deeper pipelines
 - No intra-vector dependencies → no hardware interlocking within a vector
 - No control flow within a vector
 - Known stride allows prefetching of vectors into cache/memory

Vector Processor Advantages



- + No dependencies within a vector
 - Pipelining, parallelization work well
 - Can have very deep pipelines, no dependencies!
- + Each instruction generates a lot of work
 - Reduces instruction fetch bandwidth
- + Highly regular memory access pattern
 - Interleaving multiple banks for higher memory bandwidth
 - Prefetching
- + No need to explicitly code loops
 - Fewer branches in the instruction sequence

Vector Processor Limitations



- Works (only) if parallelism is regular (data/SIMD parallelism)

- ++ Vector operations

- Very inefficient if parallelism is irregular

- How about searching for a key in a linked list?

To program a vector machine, the compiler or hand coder must make the data structures in the code fit nearly exactly the regular structure built into the hardware. That's hard to do in first place, and just as hard to change. One tweak, and the low-level code has to be rewritten by a very smart and dedicated programmer who knows the hardware and often the subtleties of the application area. Often the rewriting is

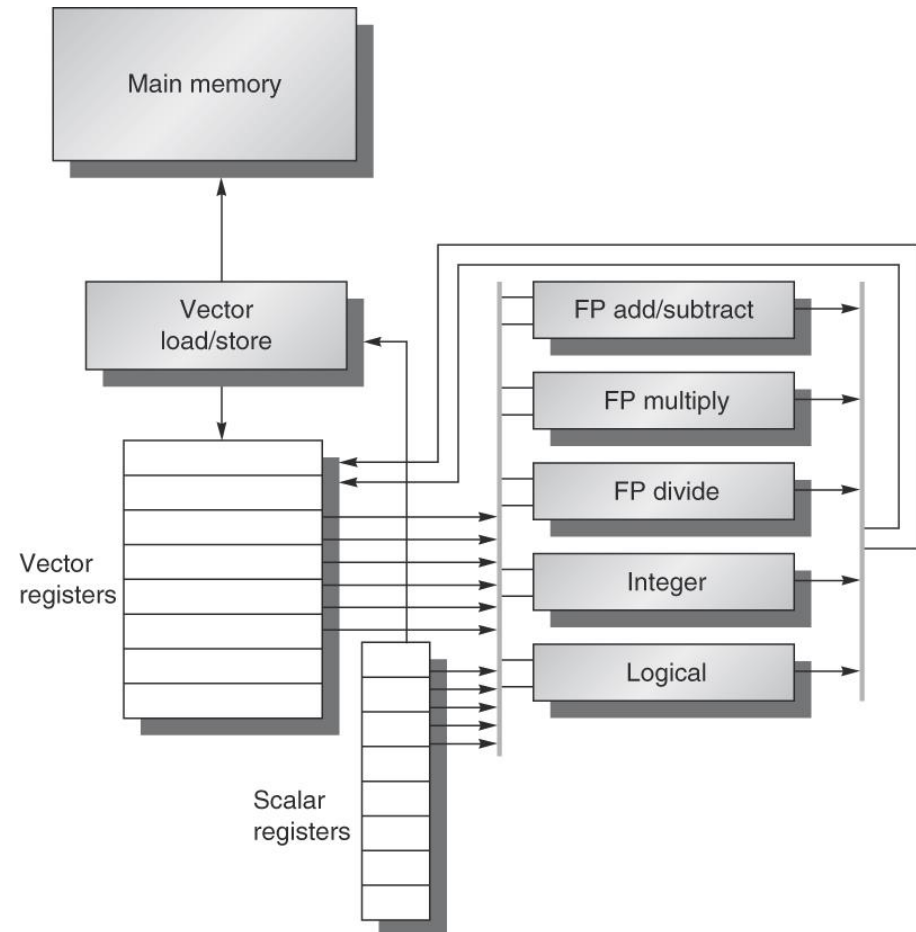
Vector Processor Limitations



- Memory (bandwidth) can easily become a bottleneck, especially if
 1. compute/memory operation balance is not maintained
 2. data is not mapped appropriately to memory banks

Vector Architectures

- Basic idea:
 - Read sets of data elements into “vector registers”
 - Operate on those registers
 - Disperse the results back into memory
- Registers are controlled by compiler
 - Used to hide memory latency
 - Leverage memory bandwidth



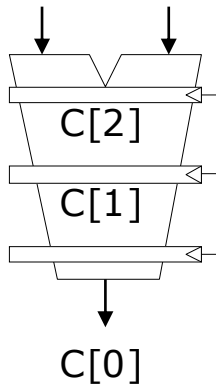
Vector Instruction Execution

ADDV C,A,B

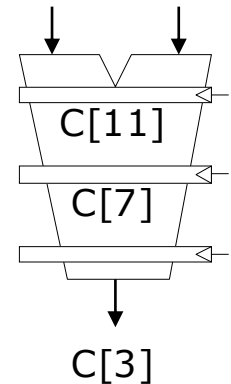
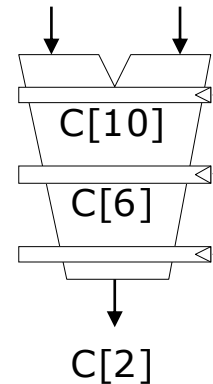
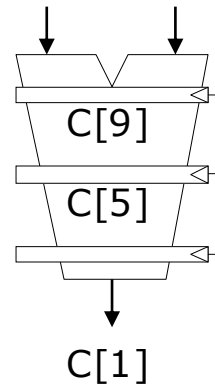
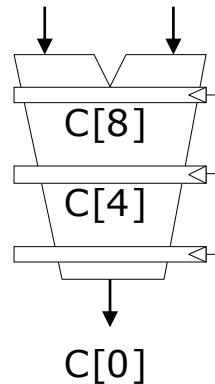
*Execution using
one pipelined
functional unit*

*Execution using
four pipelined
functional units*

A[6] B[6]
A[5] B[5]
A[4] B[4]
A[3] B[3]



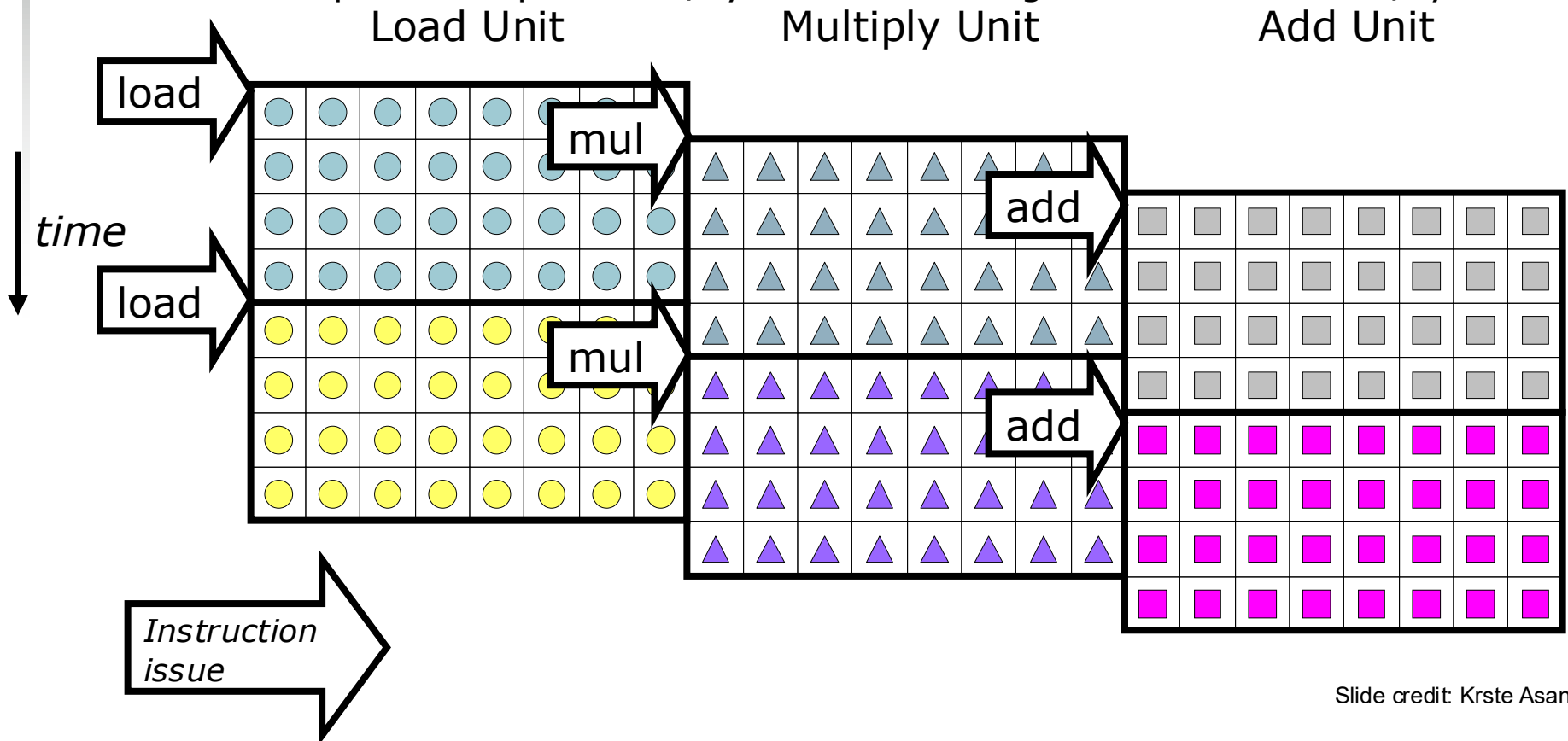
A[24] B[24] A[25] B[25] A[26] B[26] A[27] B[27]
A[20] B[20] A[21] B[21] A[22] B[22] A[23] B[23]
A[16] B[16] A[17] B[17] A[18] B[18] A[19] B[19]
A[12] B[12] A[13] B[13] A[14] B[14] A[15] B[15]



Vector Instruction Level Parallelism

Can overlap execution of multiple vector instructions

- example machine has 32 elements per vector register and 8 lanes
- Complete 24 operations/cycle while issuing 1 short instruction/cycle



Slide credit: Krste Asanovic

Vector Length Register

- Vector length not known at compile time?
- Use Vector Length Register (VLR)
- Use strip mining for vectors over the maximum vector length (MVL):

low = 0;

VL = (n % MVL); /*find odd-size piece using modulo op % */

for (j = 0; j <= (n/MVL); j=j+1) { /*outer loop*/

for (i = low; i < (low+VL); i=i+1) /*runs for length VL*/

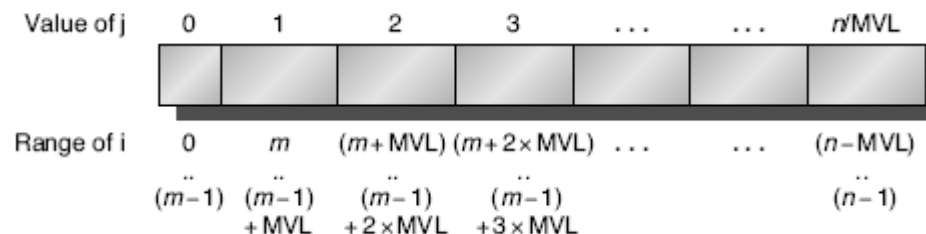
Y[i] = a * X[i] + Y[i] ; /*main operation*/

Vector operation

low = low + VL; /*start of next vector*/

VL = MVL; /*reset the length to maximum vector length*/

}



Vector Mask Registers

- Consider the loop:

```
for (i = 0; i < 32; i=i+1)
    if (X[i] != 0)
        X[i] = X[i] - Y[i];
```

- Use vector mask register, VM, to “disable” elements:

vsetpcfgi	1	;Enable 1 predicate register
vld	v0,Rx	;load vector X into V0
vld	v1,Ry	;load vector Y
fld	F0,#0	;load FP zero into F0
vpne	p0, v0, F0	;sets p0(i) to 1 if v0(i) != F0
vsub.vv	v0, v0, v1	;subtract under vector mask
vst	v0,Rx	;store the result in X
vdisable		;disable predicate registers

- GFLOPS rate decreases!

Memory Banks

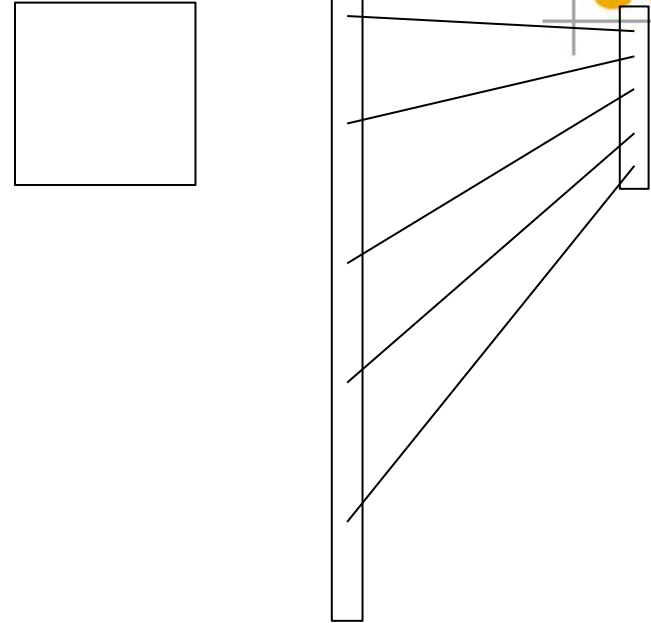
- Memory system must be designed to support high bandwidth for vector loads and stores
- Spread accesses across multiple banks
 - Control bank addresses independently
 - Load or store non sequential words
 - Support multiple vector processors sharing the same memory
- Bank-level parallelism helps.

Stride

The matrix D

D is stored
row-wise

Loading
with stride



- Consider:
for (i = 0; i < 32; i=i+1)
 for (j = 0; j < 32; j=j+1) {
 A[i][j] = 0.0;
 for (k = 0; k < 32; k=k+1)
 A[i][j] = A[i][j] + B[i][k] * D[k][j];
 }
}
- Must vectorize multiplication of rows of B with columns of D
- Use *vlds* (vector load with stride) to load the columns of a matrix into a vector.
 - `vlds v, (R1, R2)      →      v[i] = address(R1+i R2)`
- Bank conflict (stall) may occur

Scatter-Gather

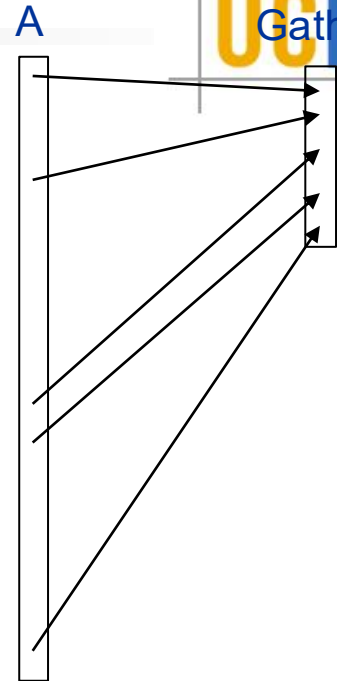
- Consider:

for ($i = 0; i < n; i=i+1$)

$A[K[i]] = A[K[i]] + C[M[i]];$

- Use *vldx* (load using index vector) , and *vstx*:

- vldx* $v1, (R1, v2) \rightarrow v1[i] = \text{address}(R1 + v2[i])$



<i>vld</i>	Vk, Rk	<i>;load K</i>
<i>vldx</i>	$Va, (Ra, Vk)$	<i>;load A[Ra+K[]] -- Gather</i>
<i>vld</i>	Vm, Rm	<i>;load M</i>
<i>vldx</i>	$Vc, (Rc, Vm)$	<i>;load C[Rc+M[]]</i>
<i>vadd.vv</i>	Va, Va, Vc	<i>;add them</i>
<i>vstx</i>	$Va, (Ra, Vk)$	<i>;store A[Ra+K[]] -- Scatter</i>

Loop-Level Parallelism (Sec. 4.5)

- Focuses on determining whether data accesses in later iterations are dependent on data values produced in earlier iterations
 - Loop-carried dependence

- Example 1:

```
for (i=0 ; i<1000 ; i++)
```

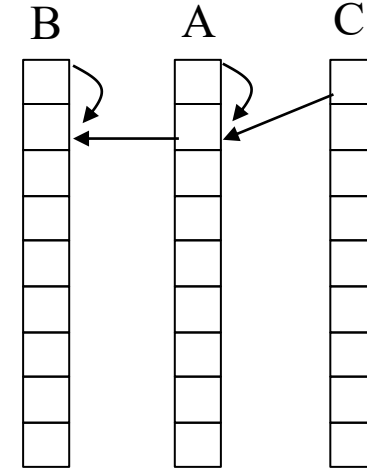
```
    x[i] = x[i] + s;           // No loop-carried dependence
```

```
for (i=2 ; i<1000 ; i++)
```

```
    x[i] = x[i-1] + x[i-2];    // loop-carried dependence
```

- Example 2:

```
for (i=0; i<100; i=i+1) {  
    A[i+1] = A[i] + C[i];    /* S1 */  
    B[i+1] = B[i] + A[i+1];  /* S2 */  
}
```

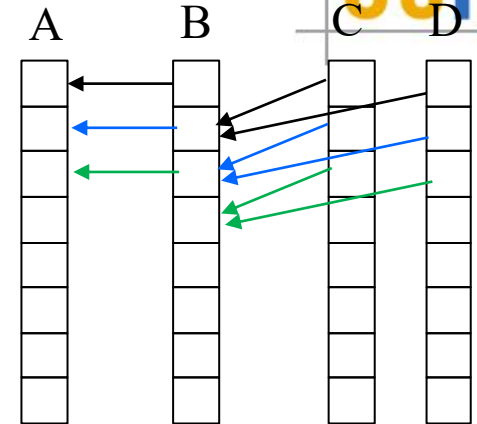


- S1 and S2 use values computed in previous iteration (bad for parallelism)
- S2 uses value computed by S1 in same iteration
- Iterations i , $i+1$, $i+2$, ... of the loop cannot be executed in parallel (non-parallelisable loop).

Loop-Level Parallelism

- Example 3:

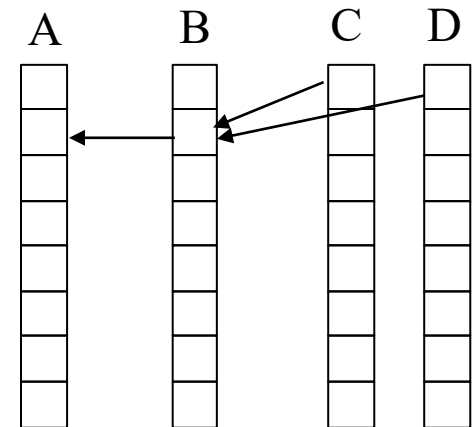
```
for (i=0; i<100; i=i+1) {  
    A[i] = A[i] + B[i];          /* S1 */  
    B[i+1] = C[i] + D[i];       /* S2 */  
}
```



- S1 uses value computed by S2 in previous iteration but dependence is not circular so loop is parallel

- May transform to a parallel loop:

```
A[0] = A[0] + B[0];  
for (i=0; i<99; i=i+1) do in parallel {  
    B[i+1] = C[i] + D[i];  
    A[i+1] = A[i+1] + B[i+1];  
}  
B[100] = C[99] + D[99];
```



- Example 4:

```
for (i=0;i<100;i=i+1) {  
    A[i] = B[i] + C[i];  
    D[i] = A[i] * E[i];  
}
```

- Example 5:

```
for (i=1;i<100;i=i+1) {  
    Y[i] = Y[i-1] + Y[i];  
}
```

- Example 6:

```
for (i=0; i<100; i=i+1) {  
    A[2 * i + 1] = B[i] + C[i];  
    D[2 * i] = A[2 * i] + E[i];  
}
```

- Example 6 is a special case of the following:

```
for (i=1; i<100; i=i+1) {  
    A[a * i + b] = ....  
    D[i] = A[c * i + d] + ....  
}
```

- Example 2:

```
for (i=0; i<100; i=i+1) {  
    Y[i] = X[i] / c; /* S1 */  
    X[i] = X[i] + c; /* S2 */  
    Z[i] = Y[i] + c; /* S3 */  
    Y[i] = c - Y[i]; /* S4 */  
}
```

- Watch for anti-dependencies and output dependencies